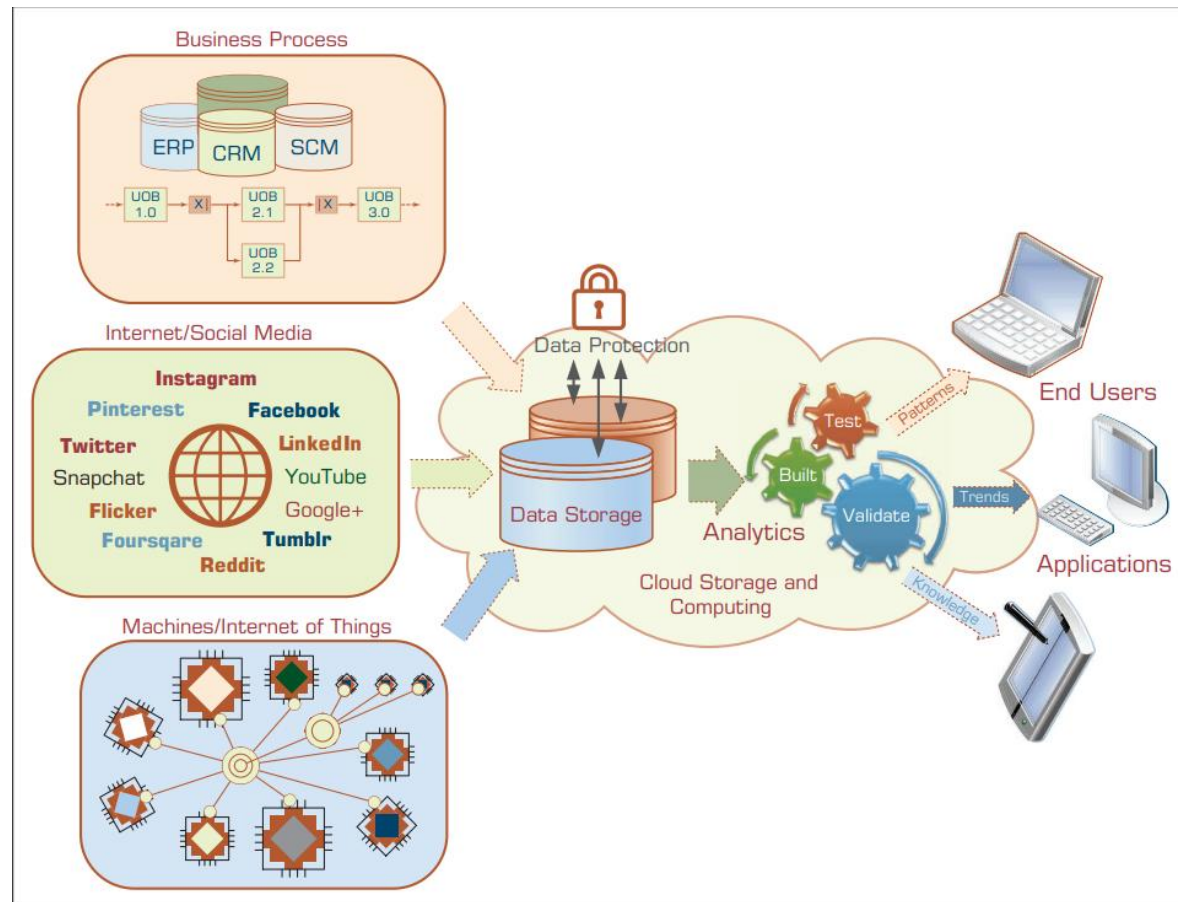


2.1 The Nature of Data

Data is the main ingredient for any BI, data science, and business analytics initiative. In fact, it can be viewed as the raw material for what these popular decision technologies produce—information, insight, and knowledge.

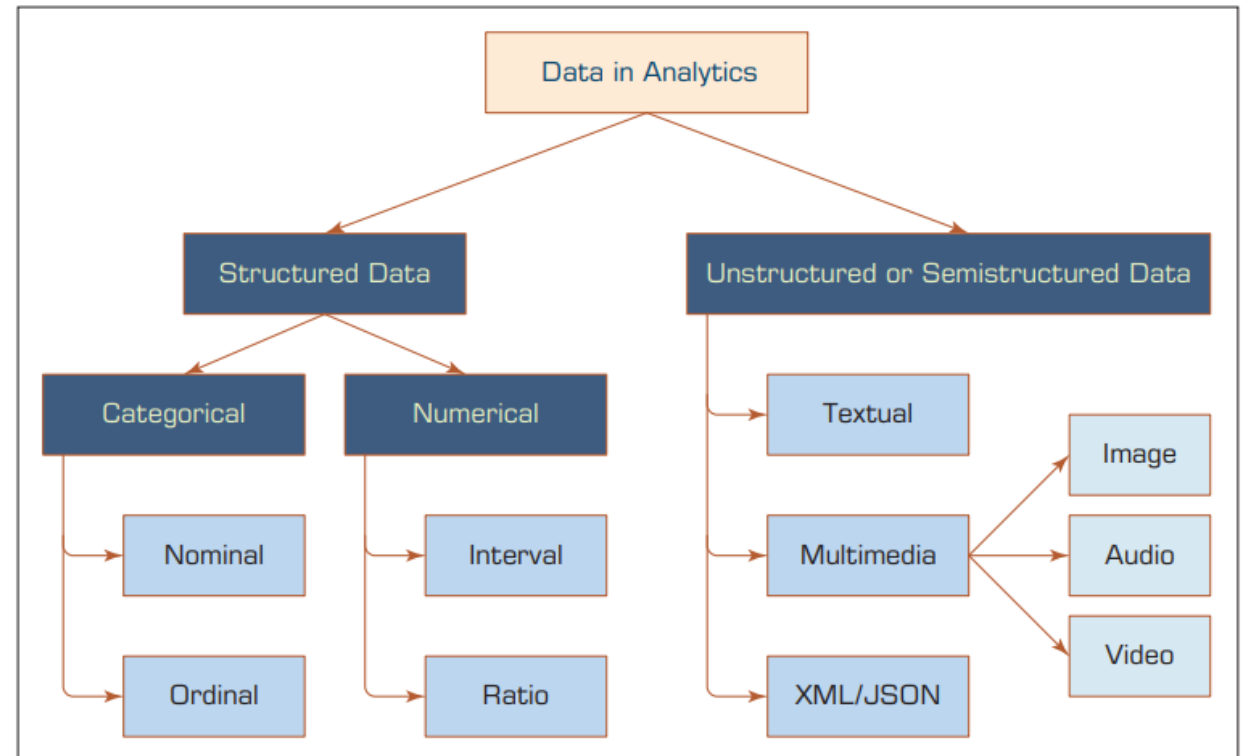


2.3 A Simple Taxonomy of Data

At the highest level of abstraction, one can classify data as structured and unstructured (or semi-structured).

Unstructured data/semi-structured data is composed of any combination of textual, imagery, voice, and Web content.

Structured data is what data mining algorithms use and can be classified as categorical or numeric.



>> Categorical data represent the labels of multiple classes used to divide a variable into specific groups.

Example. Categorical variables include race, sex, age group, and educational level.

Nominal data contain measurements of simple codes assigned to objects as labels, which are not measurements.

Example. Generally categorized as (1) single, (2) married, and (3) divorced. Nominal data can be represented with *binomial values* having two possible values (e.g., yes/no, true/ false, good/bad), or multinomial values having three or more possible values (brown/green/blue, white/black/Latino/Asian, Single/Divorced).

Ordinal data contain codes assigned to objects or events as labels that also represent the rank order among them.

Example. categorized as (1) low, (2) medium, or (3) high. Similar ordered relationships can be seen in variables such as age group (i.e., child, young, middle-aged, elderly) and educational level (i.e., high school, college, graduate school).

>> Numeric data represent the numeric values of specific variables.

Example. of numerically valued variables include age, number of children, total household income (in U.S. dollars), travel distance (in miles), and temperature (in Fahrenheit degrees).

Numeric values representing a variable can be integer (taking only whole numbers) or real (taking also the fractional number).

The numeric data may also be called continuous data, implying that the variable contains continuous measures on a specific scale that allows insertion of interim values.

Interval data are variables that can be measured on interval scales.

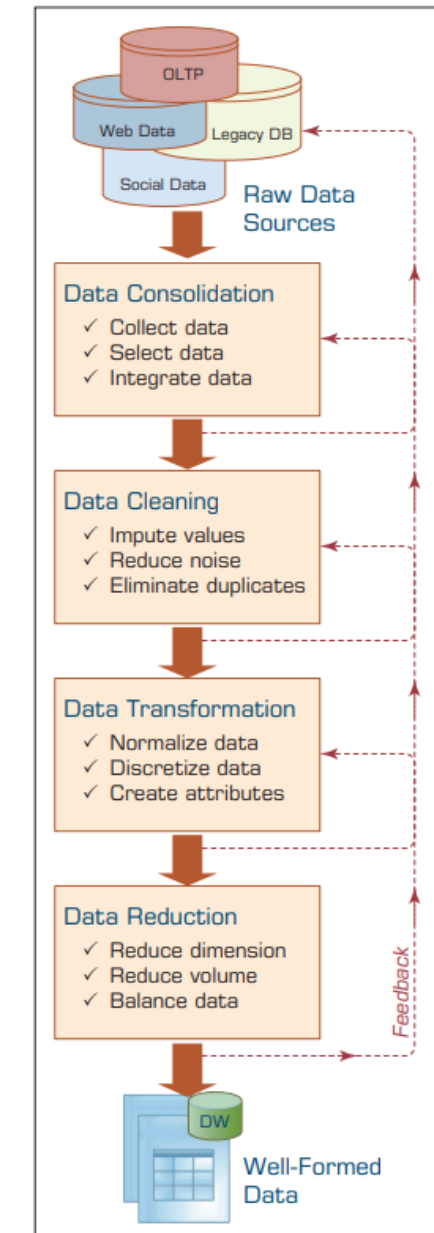
Example. Interval scale measurement is temperature on the Celsius scale.

Ratio data include measurement variables commonly found in the physical sciences and engineering. Mass, length, time, plane angle, energy, and electric charge are examples of physical measures that are ratio scales.

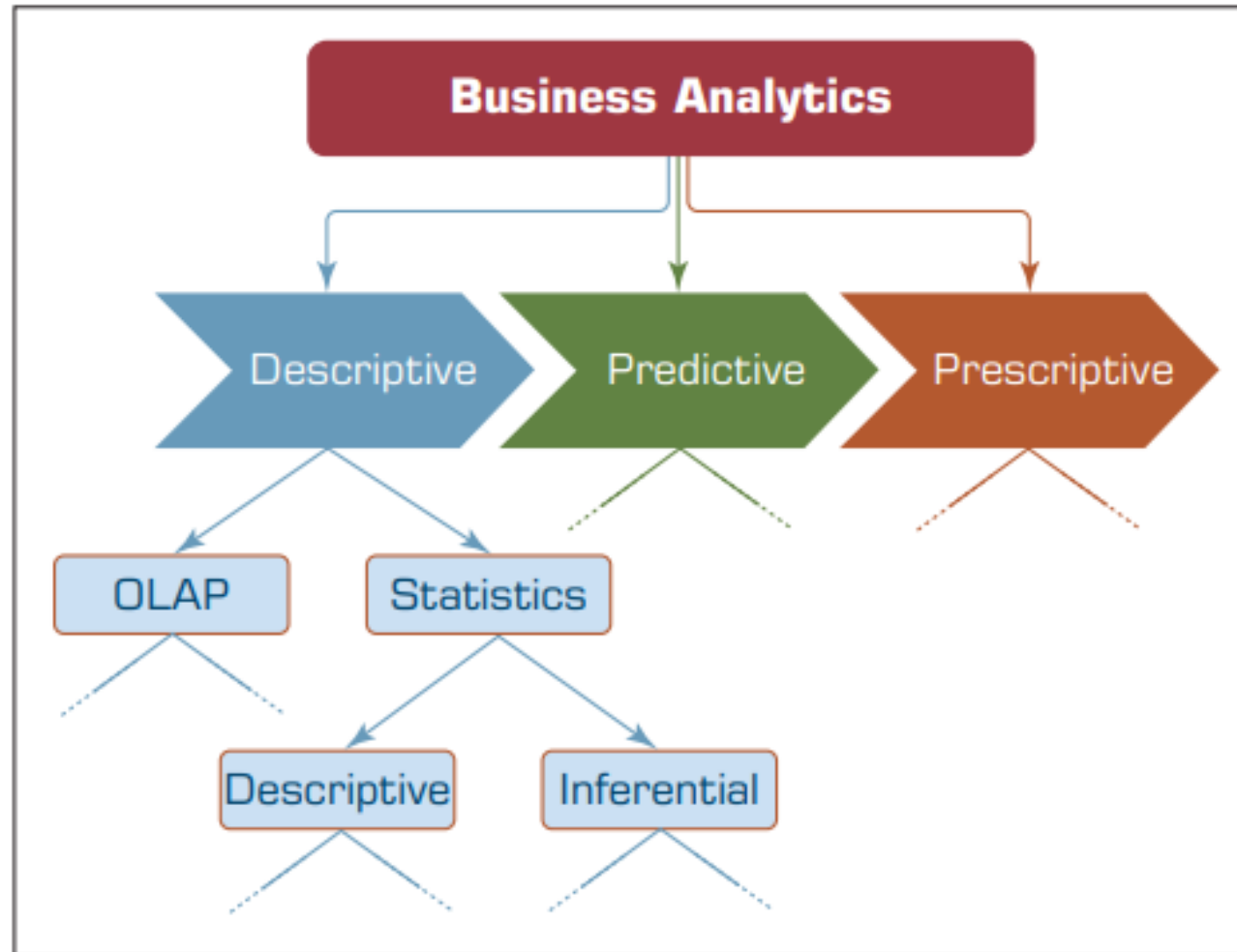
2.4 The Art and Science of Data Preprocessing

TABLE 2.1 A Summary of Data Preprocessing Tasks and Potential Methods

Main Task	Subtasks	Popular Methods
Data consolidation	Access and collect the data	SQL queries, software agents, Web services.
	Select and filter the data	Domain expertise, SQL queries, statistical tests.
	Integrate and unify the data	SQL queries, domain expertise, ontology-driven data mapping.
Data cleaning	Handle missing values in the data	Fill in missing values (imputations) with most appropriate values (mean, median, min/max, mode, etc.); recode the missing values with a constant such as "ML"; remove the record of the missing value; do nothing.
	Identify and reduce noise in the data	Identify the outliers in data with simple statistical techniques (such as averages and standard deviations) or with cluster analysis; once identified, either remove the outliers or smooth them by using binning, regression, or simple averages.
	Find and eliminate erroneous data	Identify the erroneous values in data (other than outliers), such as odd values, inconsistent class labels, odd distributions; once identified, use domain expertise to correct the values or remove the records holding the erroneous values.
Data transformation	Normalize the data	Reduce the range of values in each numerically valued variable to a standard range (e.g., 0 to 1 or -1 to +1) by using a variety of normalization or scaling techniques.
	Discretize or aggregate the data	If needed, convert the numeric variables into discrete representations using range- or frequency-based binning techniques; for categorical variables, reduce the number of values by applying proper concept hierarchies.
	Construct new attributes	Derive new and more informative variables from the existing ones using a wide range of mathematical functions (as simple as addition and multiplication or as complex as a hybrid combination of log transformations).
Data reduction	Reduce number of attributes	Principal component analysis, independent component analysis, chi-square testing, correlation analysis, and decision tree induction.
	Reduce number of records	Random sampling, stratified sampling, expert-knowledge-driven purposeful sampling.
	Balance skewed data	Oversample the less represented or undersample the more represented classes.



2.5 Statistical Modeling for Business Analytics



2.6 Regression Modeling for Inferential Statistics

Regression, especially linear regression, is perhaps the most widely known and used analytics technique in statistics.

As popular as it is, essentially, regression is a relatively simple statistical technique to model the dependence of a variable (response or output variable) on one (or more) explanatory (input) variables.

Regression can be used for one of two purposes:

hypothesis testing—investigating potential relationships between different variables.

prediction/forecasting—estimating values of a response variables based on one or more explanatory variables.

- **Difference between Correlation and regression**
- **Applications of both Correlation and regression**
- **Examples of both Correlation and regression**

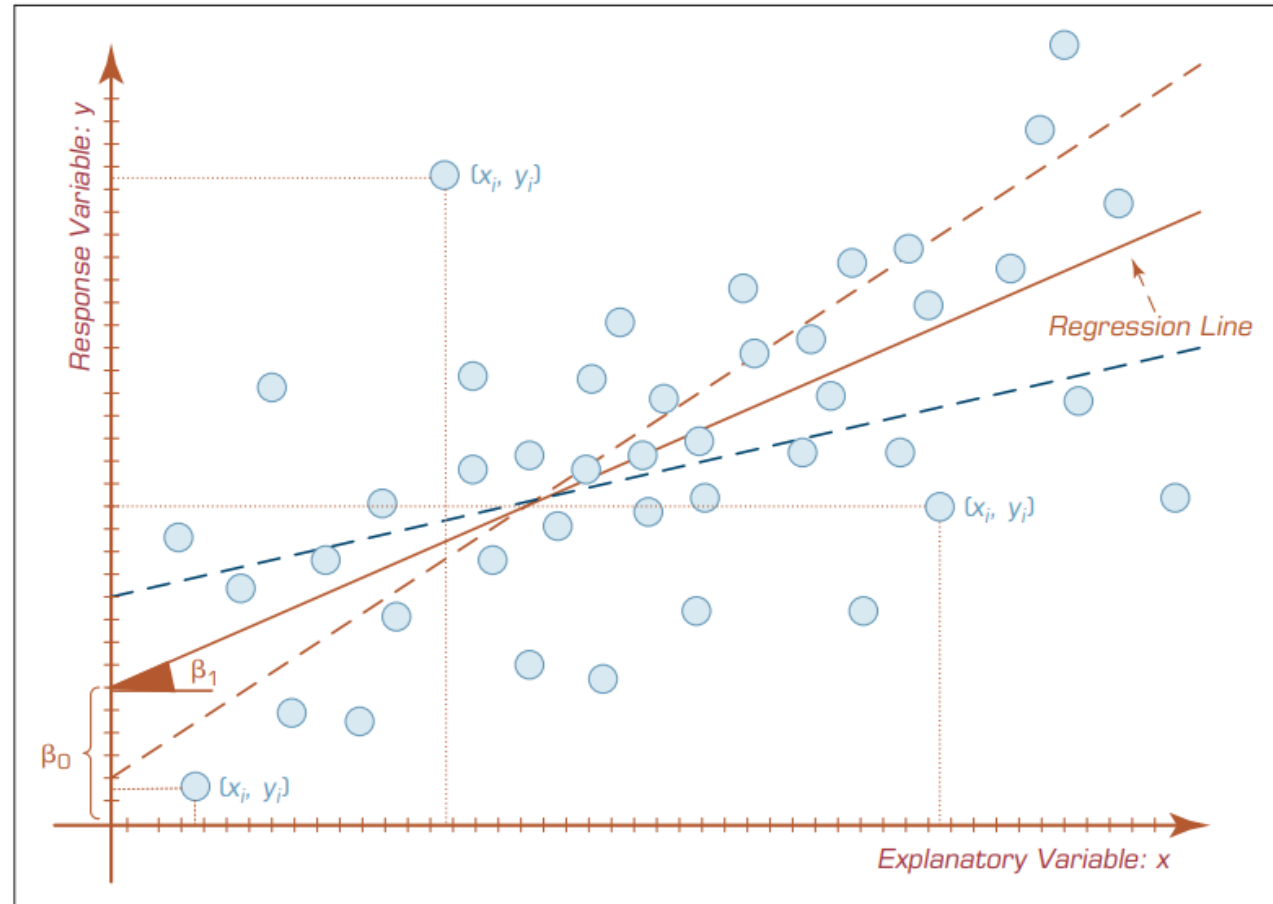
Simple versus Multiple Regression

If the regression equation is built between **one response variable and one explanatory variable**, then it is called **simple regression**.

Example. the regression equation built to predict/explain the relationship between a height of a person (explanatory variable) and the weight of a person (response variable) is a good example of simple regression.

Multiple regression is the extension of simple regression where the explanatory variables are more than one.

Example. in the previous example, if we were to include not only the height of the person but also other personal characteristics (e.g., BMI, gender, ethnicity) to predict the weight of a person, then it is called multiple regression analysis.



Even though there are several methods/algorithms proposed to identify the regression line, the one that is most commonly used is called the **ordinary least squares (OLS) method**.

Important example to understand the Model.

For example, it can be used to predict the relationship between reckless driving and the total number of road accidents caused by a driver, or, to use a business example, the effect on sales and spending a certain amount of money on advertising.

Regression is one of the most common models of machine learning.

How Do We Know If the Model Is good enough?

Sometimes models as representations of the reality do not prove to be good. Regardless of the number of explanatory variables included, there is always a possibility of not having a good model.

In the simplest sense, a well-fitting regression model results in predicted values close to the observed data values.

For the numerical assessment, three statistical measures are often used in evaluating the fit of a regression model.

- R^2 (R-squared)
- the overall F-test
- the root mean square error (RMSE)

All three of these measures are based on the sums of the square errors (*how far the data are from the mean and how far the data are from the model's predicted values*).

Of the three:

R^2 has the most useful and understandable meaning because of its intuitive scale.

The value of **R^2** ranges from **zero to one** (corresponding to the amount of variability explained in percentage) with zero indicating that the relationship and the prediction power of the proposed model is not good, and one indicating that the proposed model is a perfect fit that produces exact predictions (which is almost never the case).

The good **R^2** values would usually come close to one, and the closeness is a matter of the phenomenon being modeled.

For example. whereas an **R^2** value of 0.3 for a linear regression model in social sciences can be considered good enough, an **R^2** value of 0.7 in engineering may be considered as not a good-enough fit.

What Are the Most Important Assumptions in Linear Regression? (self study)

- Linearity
- Independence
- Normality
- Constant variance
- Multicollinearity

Logistic Regression

Logistic regression (LR) is a very popular, statistically sound, probability-based classification algorithm that employs supervised learning. It has been used extensively in **numerous disciplines**, including the **medical and social sciences fields**.

LR differs from linear regression with one major point: its output (response variable) is a class as opposed to a numerical variable. **LR** is used to classify a categorical variable.

Even though the original form of **LR** was developed for a binary output variable (e.g., 1/0, yes/no, pass/fail, accept/reject), the present-day modified version is capable of predicting multiclass output variables (i.e., multinomial **LR**).

In predictive analytics, **LR** models are used to develop probabilistic models between one or more explanatory/predictor variables (which may be a mix of both continuous and categorical in nature) and a class/response variable (which may be binomial/binary or multinomial/multiclass)

2.7 Business Reporting

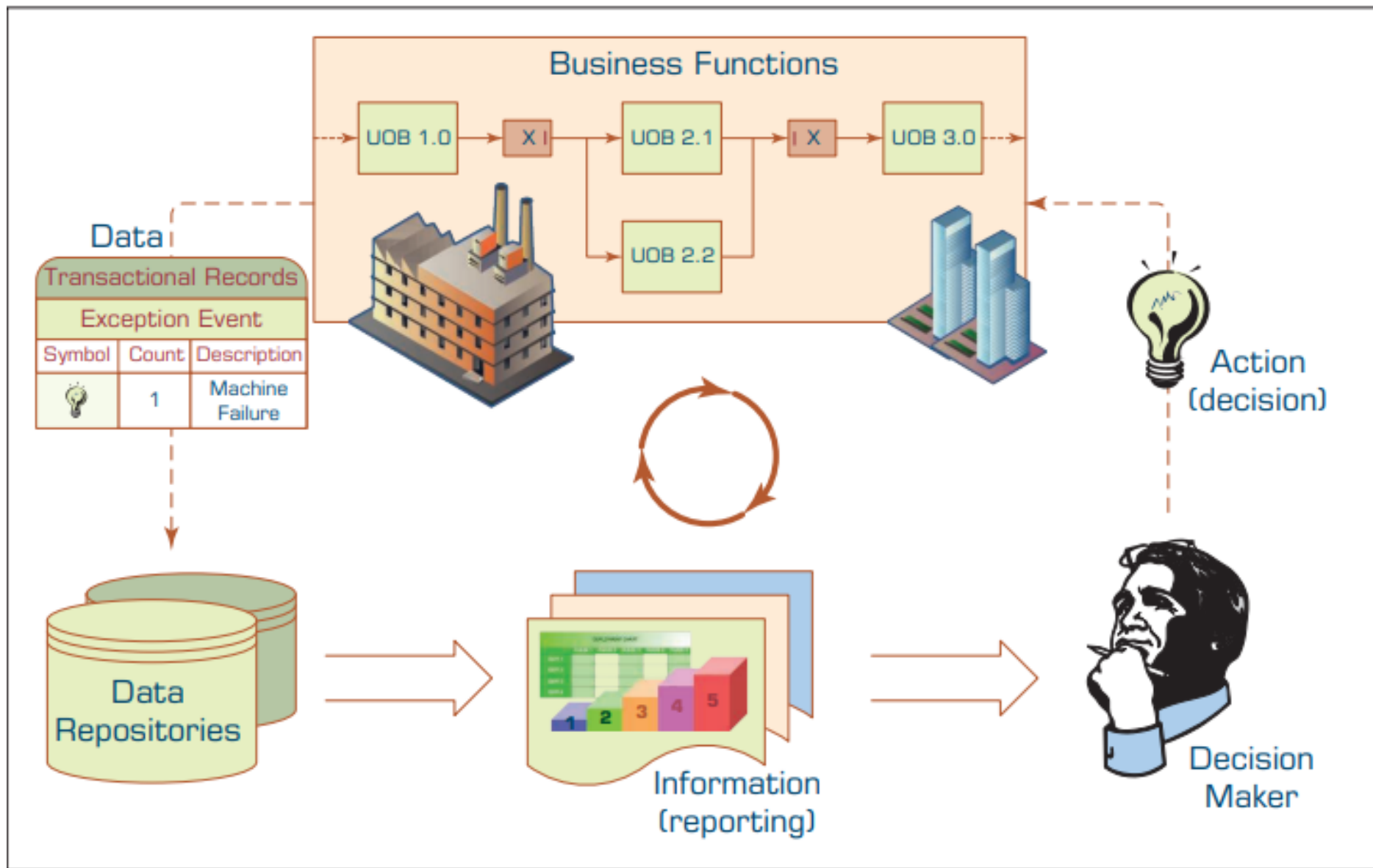
Decision makers are in need of information to make accurate and timely decisions.

Information is essentially the contextualization of data. In addition to statistical means, information (descriptive analytics) can also be obtained using online analytics processing [OLTP] systems.

Business reporting (also called OLAP or BI) is an essential part of the larger drive toward improved, evidence-based, optimal managerial decision making.

The foundation of these business reports is various sources of data coming from both inside and outside the organization (online transaction processing [OLTP] systems).

Creation of these reports involves **ETL** (extract, transform, and load) procedures in coordination with a data warehouse and then using one or more reporting tools.



2.8 Data Visualization

Data visualization (information visualization) has been defined as “the use of visual representations to explore, make sense of, and communicate data”. Although the name that is commonly used is data visualization, usually what is meant by this is **information visualization (IR)**.

Because information is the aggregation, summarization, and contextualization of data (raw facts), what is portrayed in visualizations is the information and not the data.

2.9 Different Types of Charts and Graphs

Self study

2.10 The Emergence of Visual Analytics

As Seth Grimes (2009a,b) has noted, there is a “growing palate” of data visualization techniques and tools that enable the users of business analytics and BI systems to better “communicate relationships, add historical context, uncover hidden correlations, and tell persuasive stories that clarify and call to action.”

In BI and analytics, the key challenges for visualization have revolved around the intuitive representation of large, complex data sets with multiple dimensions and measures.

For the most part, the typical charts, graphs, and other visual elements used in these applications usually involve two dimensions, sometimes three, and fairly small subsets of data sets. In contrast, the data in these systems reside in a data warehouse.

At a minimum these warehouses involve a range of dimensions (e.g., product, location, organizational structure, time), a range of measures, and millions of cells of data. In an effort to address these challenges, a number of researchers have developed a variety of new visualization techniques.

2.11 Information Dashboards

Information dashboards are common components of most, if not all, BI or business analytics platforms, business performance management systems, and performance measurement software suites.

Dashboards provide visual displays of important information that is consolidated and arranged on a single screen so that information can be digested at a single glance and easily drilled in and further explored.

Dashboard Design Dashboards are not a new concept.

According to Eckerson (2006), a well-known expert on BI in general and dashboards in particular has three layers of information:

1. **Monitoring:** Graphical, abstracted data to monitor key performance metrics.
2. **Analysis:** Summarized dimensional data to analyze the root cause of problems.
3. **Management:** Detailed operational data that identify what actions to take to resolve a problem

Go for Anaconda

Click the free download



[Products](#) [Solutions](#) [Resources](#) [Partners](#) [Company](#)

 [Free Download](#)

[Sign Up](#)

[Sign In](#)

Take the State of Data Science & AI survey!

Enter for a chance to win a \$100 Amazon gift card.

[Learn More](#)

The Operating System for AI

The world's most trusted open ecosystem for
sourcing, building, and deploying data science and AI initiatives

[Explore Anaconda Hub >](#)

[Create Account >](#)



Distribution

Free Download*

Register to get everything you need to get started on your workstation including Cloud Notebooks, Navigator, AI Assistant, Learning and more.

- ✓ Easily search and install thousands of data science, machine learning, and AI packages
- ✓ Manage packages and environments from a desktop application or work from the command line
- ✓ Deploy across hardware and software platforms
- ✓ Distribution installation on Windows, MacOS, or Linux

Provide email to download Distribution

Email Address: **Put your email address**

☐

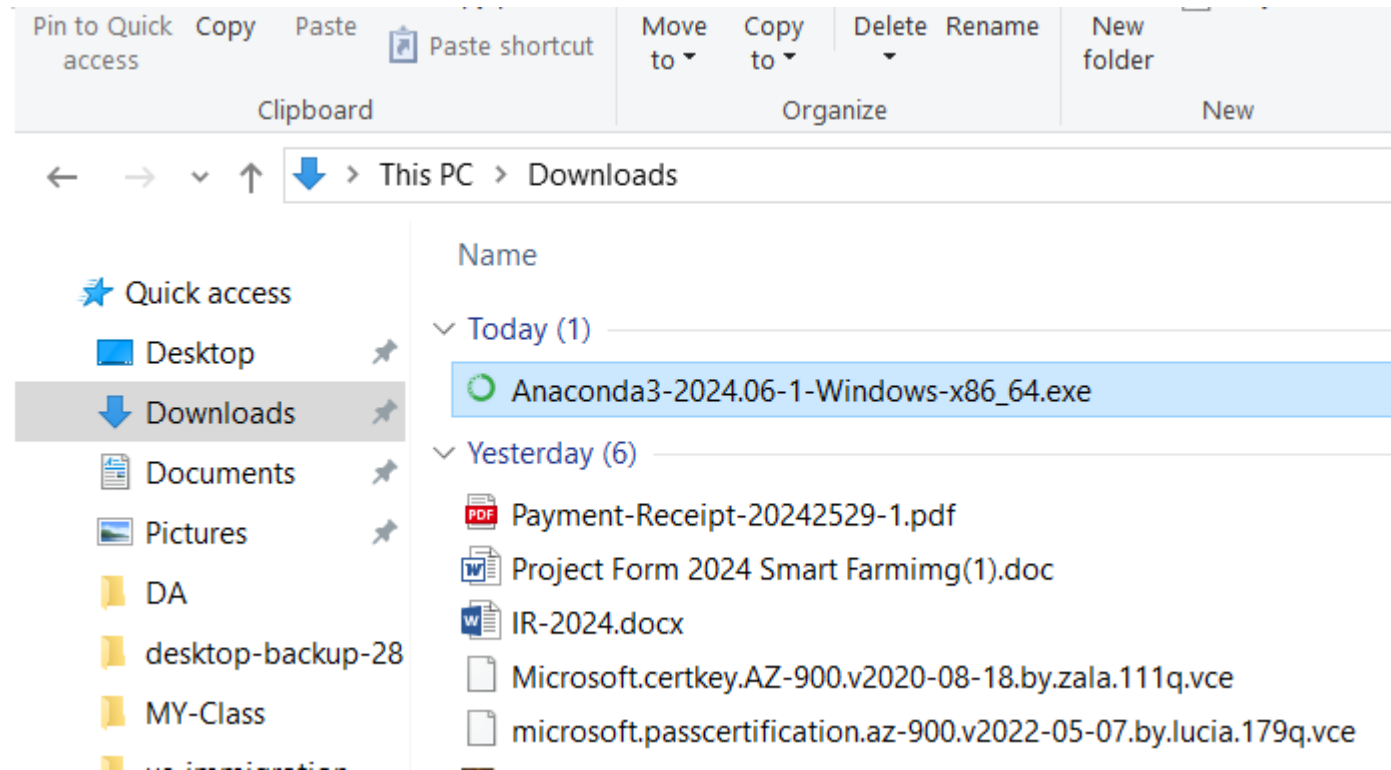
Agree to receive communication from Anaconda regarding relevant content, products, and services. I understand that I can revoke this consent [here](#) at any time.

By continuing, I agree to Anaconda's [Privacy Policy](#) and [Terms of Service](#).

Submit >

[Skip registration](#)

Go to email and click the link and go for download



Anaconda3 2024.06-1 (64-bit) Setup



Welcome to Anaconda3 2024.06-1 (64-bit) Setup

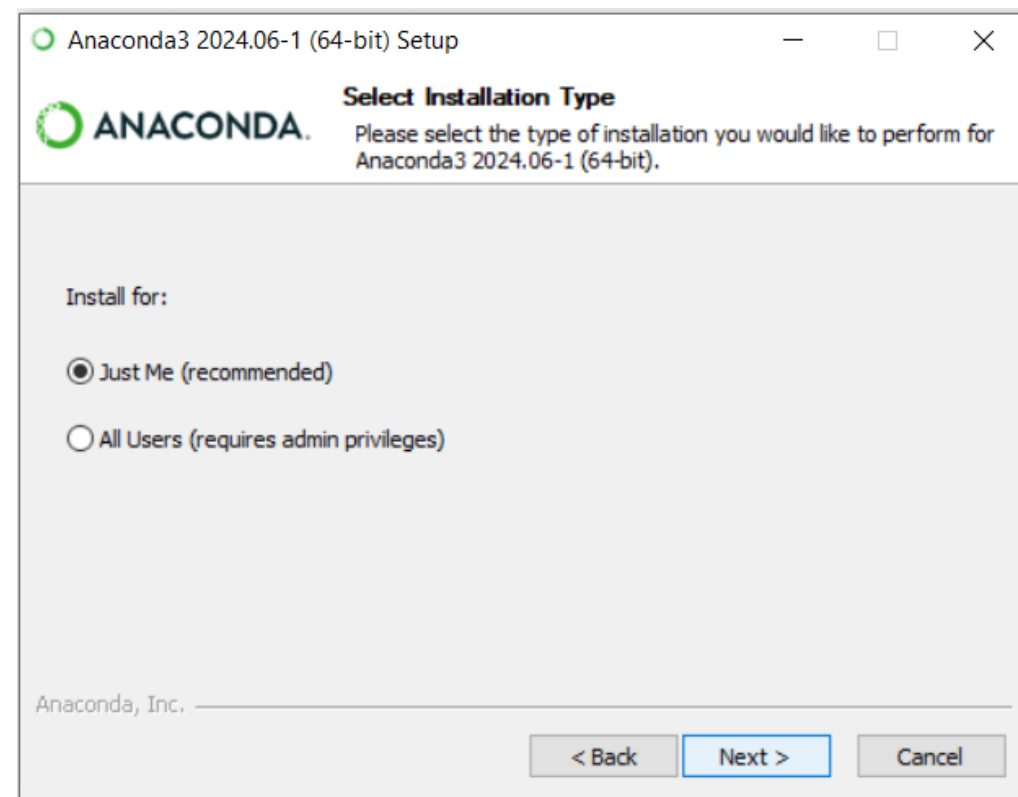
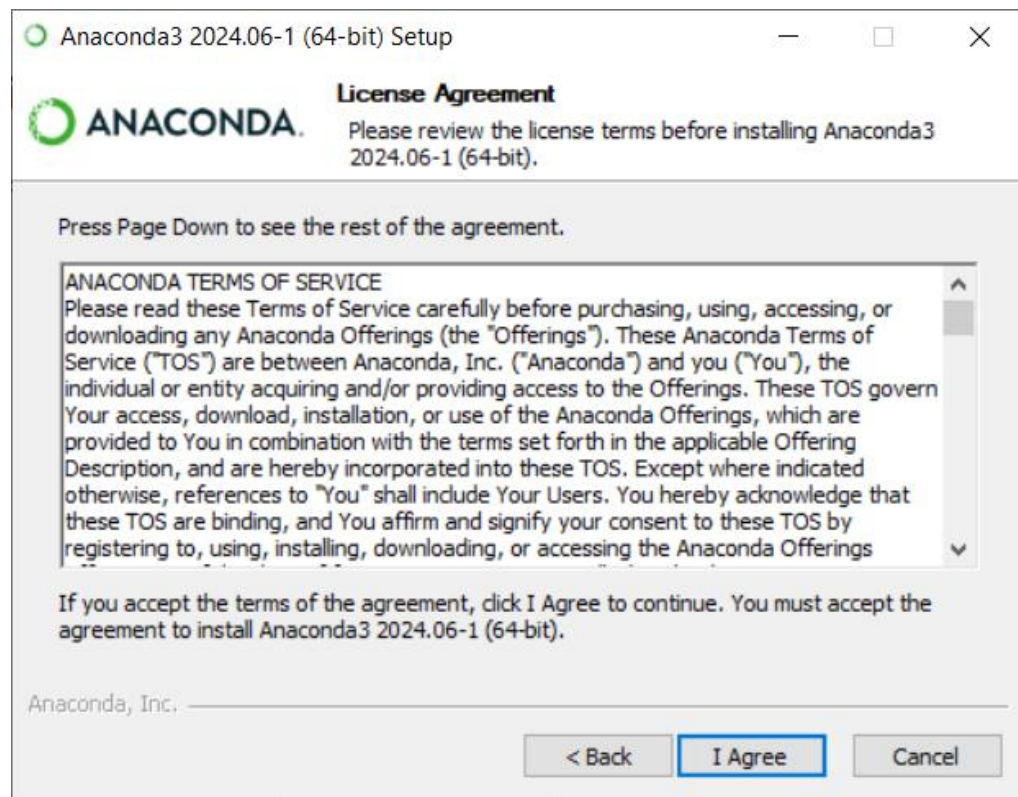
Setup will guide you through the installation of Anaconda3 2024.06-1 (64-bit).

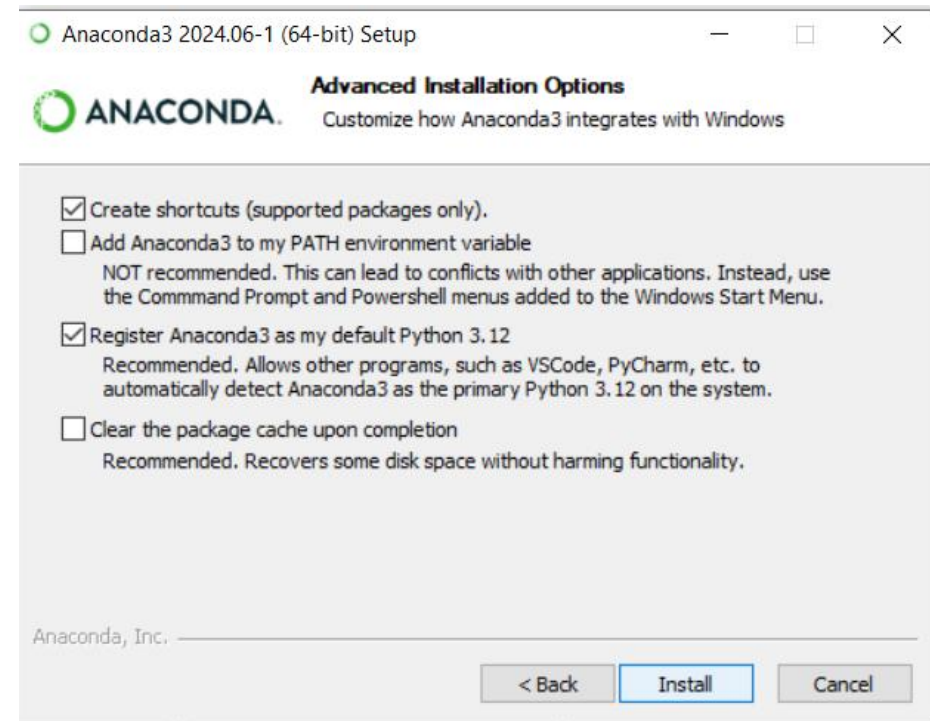
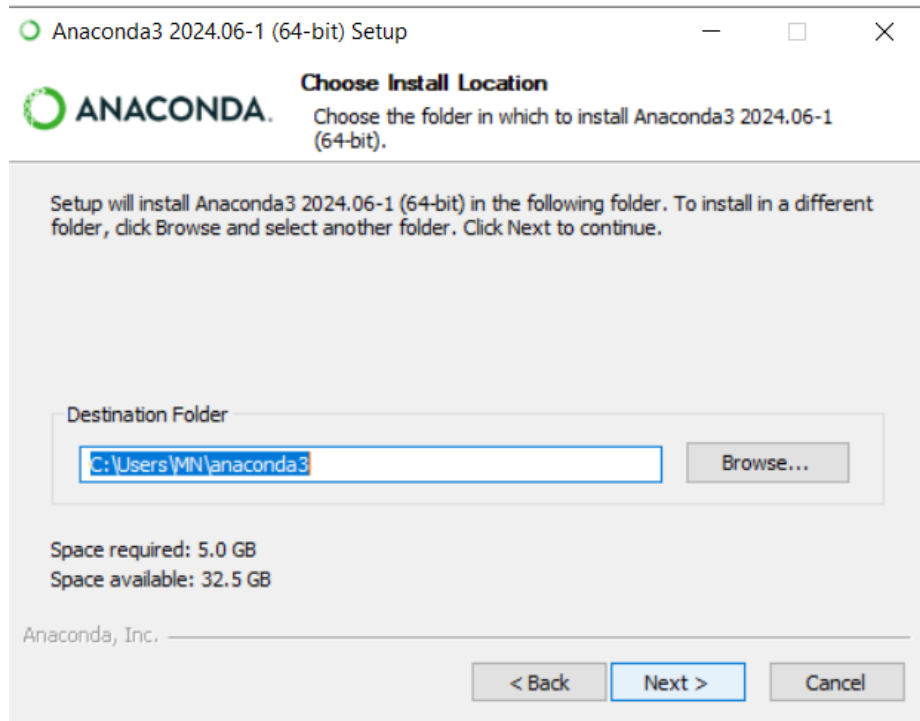
It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.

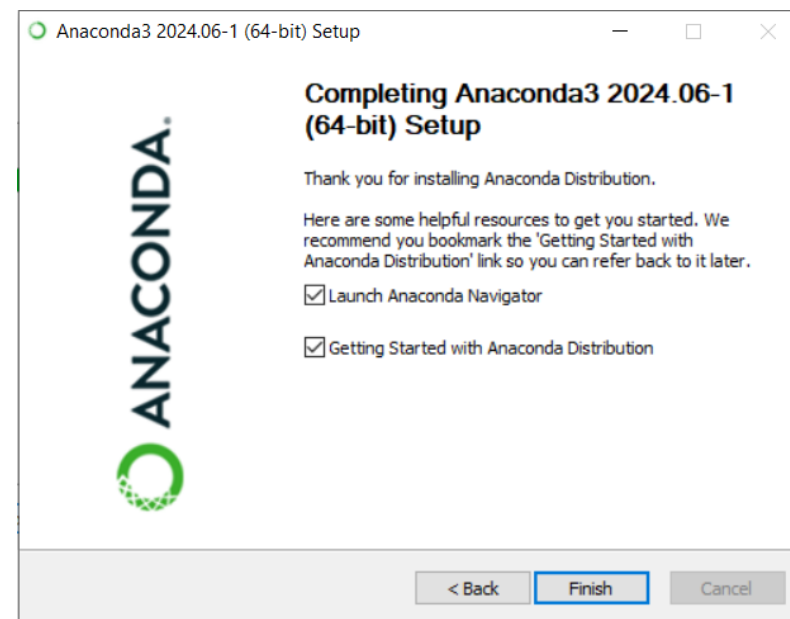
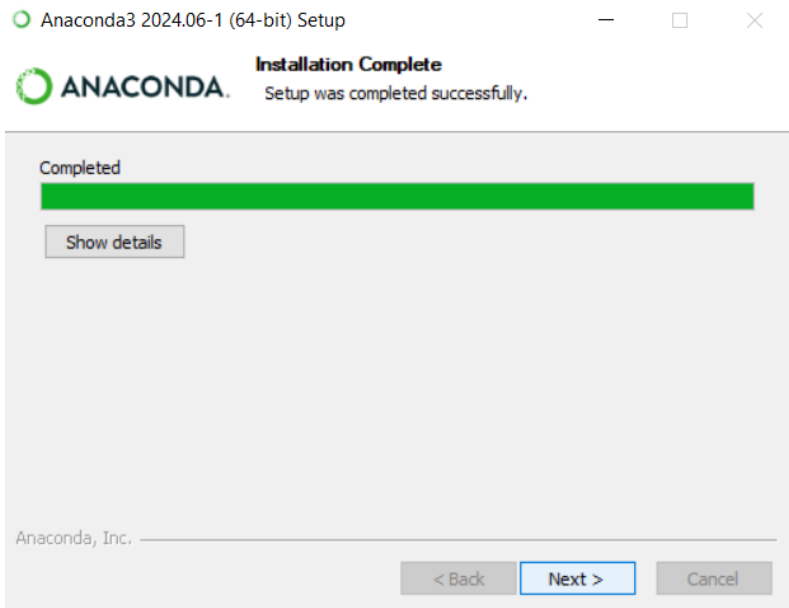
Click Next to continue.

Next >

Cancel







Best match

>_

Anaconda Prompt

App

Anaconda Navigator

App

>

Apps

Anaconda3-2024.06-1-Windows-x86_64.exe

>

Anaconda Powershell Prompt

>

Search the web

anaconda - See more search results

>

anaconda download

>

anaconda navigator

>

Folders

@anaconda - in schemas

>

@anaconda - in labextensions

>

@anaconda - in schemas

>

@anaconda - in schemas

>

anaconda Prompt

>

>_

Anaconda Prompt

App

Open

Run as administrator

Open file location

Pin to Start

Pin to taskbar

Uninstall

New

Upload

Last Modified	File Size
5 months ago	
4 minutes ago	
3 months ago	
5 months ago	
5 minutes ago	
9 minutes ago	
40 minutes ago	
5 months ago	
5 months ago	
5 months ago	
5 months ago	
5 months ago	
5 months ago	
4 months ago	
5 months ago	

zoom

>_

Create a folder “JupyterCode”
that we can save out python program

Anaconda Prompt

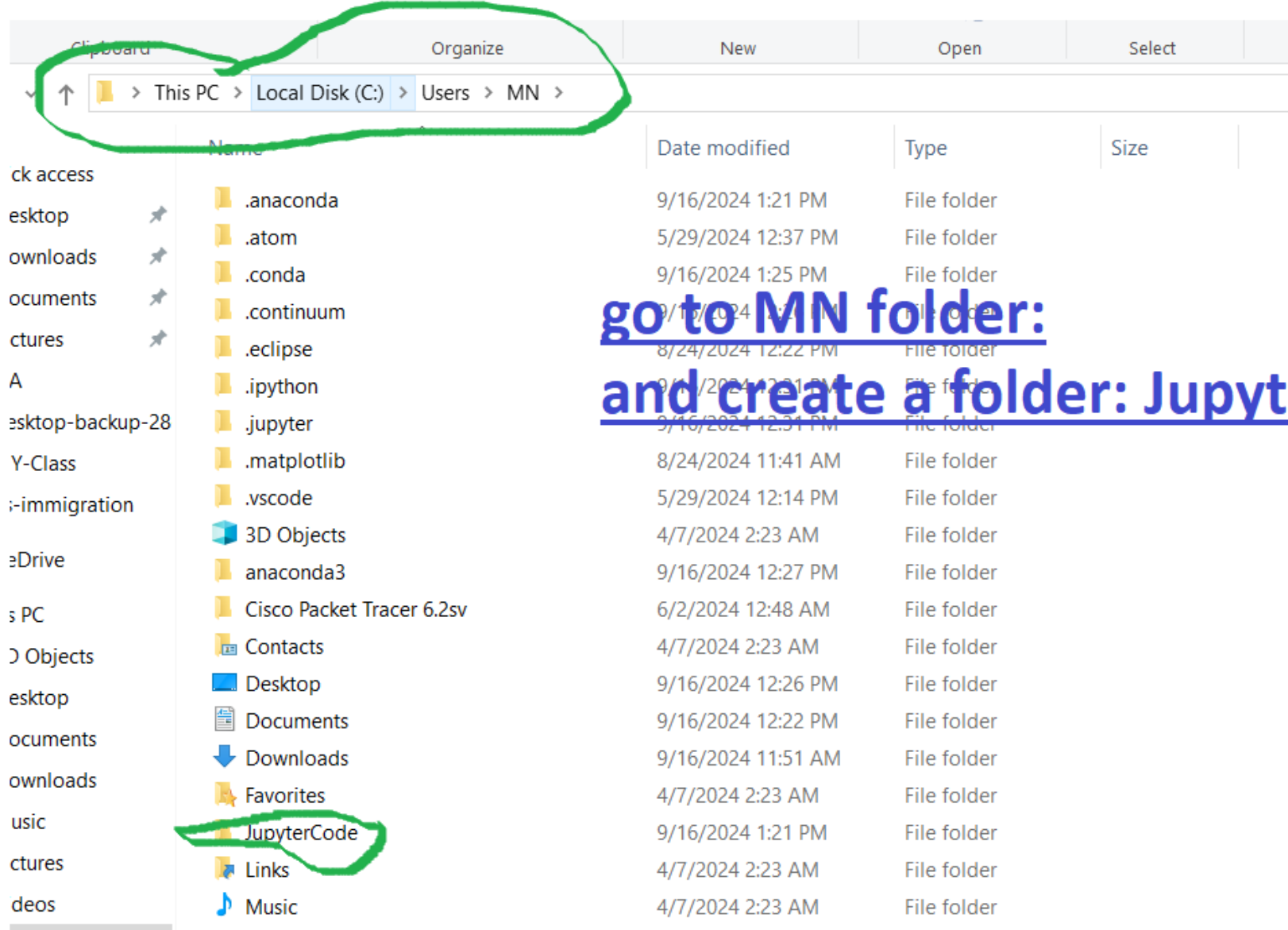
```
(base) C:\Users\MN>mkdir JupyterCode
```

```
(base) C:\Users\MN>jupyter notebook
```

```
[I 2024-09-16 12:31:46.910 ServerApp] Extension package aext_assistant took 0.3711s to import
[I 2024-09-16 12:31:46.934 ServerApp] ***** ENVIRONMENT Environment.PRODUCTION *****
[I 2024-09-16 12:31:46.950 ServerApp] ***** ENVIRONMENT Environment.PRODUCTION *****
[W 2024-09-16 12:31:47.012 ServerApp] A `_jupyter_server_extension_points` function was not found in jupyter_lsp. Instead, a `_jupyter_server_extension_paths` function was found and will be used for now. This function name will be deprecated in future releases of Jupyter Server.
[W 2024-09-16 12:31:47.156 ServerApp] A `_jupyter_server_extension_points` function was not found in notebook_shim. Instead, a `_jupyter_server_extension_paths` function was found and will be used for now. This function name will be deprecated in future releases of Jupyter Server.
[I 2024-09-16 12:31:48.624 ServerApp] Extension package panel.io.jupyter_server_extension took 1.4668s to import
[I 2024-09-16 12:31:48.624 ServerApp] aext_assistant | extension was successfully linked.
[I 2024-09-16 12:31:48.624 ServerApp] aext_core | extension was successfully linked.
[I 2024-09-16 12:31:48.624 ServerApp] aext_panels | extension was successfully linked.
[I 2024-09-16 12:31:48.624 ServerApp] aext_share_notebook | extension was successfully linked.
[I 2024-09-16 12:31:48.624 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2024-09-16 12:31:48.640 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2024-09-16 12:31:48.645 ServerApp] jupyterlab | extension was successfully linked.
[I 2024-09-16 12:31:48.657 ServerApp] notebook | extension was successfully linked.
[I 2024-09-16 12:31:48.662 ServerApp] Writing Jupyter server cookie secret to C:\Users\MN\AppData\Roaming\jupyter\runtime\jupyter_cookie_secret
[I 2024-09-16 12:31:49.310 ServerApp] notebook_shim | extension was successfully linked.
[I 2024-09-16 12:31:49.310 ServerApp] panel.io.jupyter_server_extension | extension was successfully linked.
[I 2024-09-16 12:31:49.366 ServerApp] notebook_shim | extension was successfully loaded.
[I 2024-09-16 12:31:49.366 ServerApp] Registered aext_assistant server extension
[I 2024-09-16 12:31:49.366 ServerApp] aext_assistant | extension was successfully loaded.
[I 2024-09-16 12:31:49.381 ServerApp] Registered aext_core server extension
[I 2024-09-16 12:31:49.381 ServerApp] aext_core | extension was successfully loaded.
[I 2024-09-16 12:31:49.381 ServerApp] Registered aext_panels server extension
```



```
file:///C:/Users/MN/AppData/Roaming/jupyter/runtime/jpserver-8764-open.html
Or copy and paste one of these URLs:
http://localhost:8888/tree?token=d808b6ef206706cf38a1d434292f5db3dad4a3e129cdaa42
http://127.0.0.1:8888/tree?token=d808b6ef206706cf38a1d434292f5db3dad4a3e129cdaa42
[I 2024-09-16 12:31:49.607 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-
nodejs, javascript-typescript-langserver, jedi-language-server, julia-language-server, pyright, python-language-server,
r-languageserver, sql-language-server, texlab, typescript-language-server, unified-language-server, vscode-css-languages
erver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yaml-language-server
0.00s - Debugger warning: It seems that frozen modules are being used, which may
0.00s - make the debugger miss breakpoints. Please pass -Xfrozen_modules=off
0.00s - to python to disable frozen modules.
0.00s - Note: Debugging will proceed. Set PYDEVD_DISABLE_FILE_VALIDATION=1 to disable this validation.
[W 2024-09-16 12:31:59.483 ServerApp] wrote error: 'Forbidden'
Traceback (most recent call last):
  File "C:\Users\MN\anaconda3\Lib\site-packages\tornado\web.py", line 1790, in _execute
    result = await result
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\MN\anaconda3\Lib\site-packages\aext_assistant_server\handlers.py", line 117, in get
    raise HTTPError(403, reason="missing nucleus_token")
tornado.web.HTTPError: HTTP 403: missing nucleus_token
[W 2024-09-16 12:31:59.490 ServerApp] 403 GET /aext_assistant_server/nucleus_token?1726468318610 (9593ee10e7964a3781881a
75994a34b7@127.0.0.1) 400.92ms referer=http://localhost:8888/tree
```

go to MN folder:
and create a folder: JupyterCode

Files Running

Rename Delete

New Upload Refresh

/

☐ Name	Last Modified	File Size
☐ 3D Objects	5 months ago	
☐ anaconda3	18 minutes ago	
☐ Cisco Packet Tracer 6.2sv	3 months ago	
☐ Contacts	5 months ago	
☐ Desktop	18 minutes ago	
☐ Documents	22 minutes ago	
☐ Downloads	54 minutes ago	
☐ Favorites	5 months ago	
☐ JupyterCode	7 minutes ago	
☐ Libraries	5 months ago	
☐ My Documents	5 months ago	
☐ Open	5 months ago	
☐ Pictures	5 months ago	

Name: JupyterCode
 Created: 9/16/24, 12:38 PM
 Modified: 9/16/24, 12:38 PM
 Writable: true

Files

Running

Rename

Delete

New

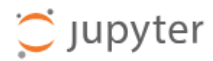
Upload

/

Name	Last Modified	File Size
☐ 3D Objects	5 months ago	
☐ anaconda3	20 minutes ago	
☐ Cisco Packet Tracer 6.2sv	3 months ago	
☐ Contacts	5 months ago	
☐ Desktop	21 minutes ago	
☐ Documents	25 minutes ago	
☐ Downloads	56 minutes ago	
☐ Favorites	5 months ago	
☒ JupyterCode	9 minutes ago	
☐ Links	5 months ago	
☐ Music	5 months ago	
☐ OneDrive	5 months ago	
☐ Pictures	5 months ago	
☐ Saved Games	5 months ago	
☐ Searches	4 months ago	
☐ Videos	5 months ago	

select and double click

Activate Windows
Go to Settings to activate Windows.



File View Settings Help

Files Running

Open Download Rename Duplicate Delete

/ JupyterCode /

✓ Name

✓ Untitled.ipynb

-> click New and select Notebook

- New
- Upload
- Refresh
- Notebook
- Terminal
- Console
- New File
- New Folder

[]:

Select Kernel

Select kernel for: "Untitled1.ipynb"

Python 3 (ipykernel) ▾

☐ Always start the preferred kernel

No Kernel

Select





```
[ ]:
```



Run this cell and advance (Shift+Enter)

```
[ ]: printf("RASHED")
```



```
[1]: printf("RASHED")
```



```
-----
NameError                                Traceback (most recent call last)
Cell In[1], line 1
----> 1 printf("RASHED")

NameError: name 'printf' is not defined
```

```
[ ]:
```

Output of our first program

```
[3]: print("RASHED")
```

RASHED

```
[ ]:
```



Another important point

```
[3]: print("RASHED")
```

RASHED

```
[ ]:
```



you will get another new interface "red mark",
you can write new program and go on.....
be an **Expert of Data Analyst**

```
[3]: print("RASHED")
```

```
RASHED
```

```
[5]: import numpy as RNA  
rashed_array = RNA.array([10, 20, 30, 40, 50])  
print(rashed_array)
```

```
[10 20 30 40 50]
```

```
[ ]:
```

Numpy library:

- NumPy is a Python library
- NumPy is used for working with arrays
- NumPy is short for "Numerical Python"

<https://www.w3schools.com/python/numpy/default.asp>


```
[3]: print("RASHED")
```

```
RASHED
```

```
[5]: import numpy as RNA
rashed_array = RNA.array([10, 20, 30, 40, 50])
print(rashed_array)
```

```
[10 20 30 40 50]
```

```
[1]: import numpy as 2_RNA
2_rm_array = 2_RNA([10, 11, 12], [21, 22, 23], [31, 32, 33])
print(2_rm_array)
```

```
Cell In[1], line 1
import numpy as 2_RNA
                ^
```

```
SyntaxError: invalid decimal literal
```

```
[ ]: |
```

Is there any other errors?

```
[3]: print("RASHED")
```

```
RASHED
```

```
[5]: import numpy as RNA
rashed_array = RNA.array([10, 20, 30, 40, 50])
print(rashed_array)
```

```
[10 20 30 40 50]
```

```
[14]: import numpy as RNA
rm_array = RNA.array([{10, 11, 12}, {21, 22, 23}, {31, 32, 33}])
print(rm_array)
```

```
[{10, 11, 12} {21, 22, 23} {32, 33, 31}]
```

Find out the shape of the matrix

```
[23]: import numpy as RNA
three_d_array = RNA.array([[[10, 11, 12],[21, 22, 23]], [[31, 32, 33], [41, 42, 43]]])
print(three_d_array.shape)

(2, 2, 3)
```

```
[2]: import numpy as RNA
      rashed_array = RNA.array([100, 200, 300, 400, 500])
      f_e = rashed_array[1]
      print(f_e)
```

200

```
[4]: import numpy as RNA
      rashed_array = RNA.array([100, 200, 300, 400, 500])
      f_e = rashed_array[1:4]
      print(f_e)
```

[200 300 400]

```
[11]: import numpy as RNA
RNA.zeros(20).reshape(5,4)
```

Problem-1
To create zero matrix

```
[11]: array([[0., 0., 0., 0.],
            [0., 0., 0., 0.],
            [0., 0., 0., 0.],
            [0., 0., 0., 0.],
            [0., 0., 0., 0.]])
```

```
[13]: import numpy as RNA
RNA.full((5,6), 99)
```

```
[13]: array([[99, 99, 99, 99, 99, 99],
            [99, 99, 99, 99, 99, 99],
            [99, 99, 99, 99, 99, 99],
            [99, 99, 99, 99, 99, 99],
            [99, 99, 99, 99, 99, 99]])
```

Problem-2

To create identity matrix

```
[15]: import numpy as RNA
      RNA.identity(5)
```

```
[15]: array([[1., 0., 0., 0., 0.],
            [0., 1., 0., 0., 0.],
            [0., 0., 1., 0., 0.],
            [0., 0., 0., 1., 0.],
            [0., 0., 0., 0., 1.]])
```

Reference. <https://docs.python.org/3/library/random.html>

```
[21]: import numpy as RNA
      p_RNA = RNA.random.rand(4,2)
      print(p_RNA)
```

```
[ [0.06405883  0.45252315]
  [0.11030007  0.97701463]
  [0.33941876  0.33858928]
  [0.71671714  0.84240548]]
```

Reference. <https://docs.python.org/3/library/random.html>

```
[29]: import numpy as RNA
p_RNA = RNA.random.randint(10, 100, 12)
print(p_RNA)
```

```
[56 16 82 45 75 52 89 16 68 47 17 34]
```

Reference. <https://www.geeksforgeeks.org/numpy-arrange-in-python/>

```
[2]: import numpy as RNA
a_RNA = RNA.arange(12)
print(a_RNA)

b_RNA = RNA.arange(2, 12, 2)
print(b_RNA)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[ 2  4  6  8 10]
```

```
import numpy as RNA
rashed_array = RNA.array([20, 30, 10, 40])
result = RNA.sum(rashed_array)
print(result)
```

100

to find the sum of all values of
declared array

```
[6]: import numpy as RNA
rashed_array = RNA.array([20, 30, 10, 40])
result = RNA.mean(rashed_array)
print(result)
```

25.0



to find out average


```
[8]: import numpy as RNA
      rashed_array = RNA.array([20, 30, 10, 40])
      result = RNA.max(rashed_array)
      print(result)
```

40



to find the max. value from
the given array

- Do yourself to find the min. value from the given array
- Find the array-index of the smallest number

```
[10]: import numpy as np
rashed_array = np.array([25, 9, 16, 100])
result = np.sqrt(rashed_array)
print(result)
```

[5. 3. 4. 10.]



to find out root square

- Find out sin, cosine, tan value

```
[17]: import numpy as RNA
      rashed_array = RNA.array([25, 9, 16, 100, 4, 36, 49])
      result = RNA.std(rashed_array)
      print(result)
```

30.507275813337063



? ? ?

```
import numpy as RNA
hehe = RNA.ones((4,4))
print(hehe)
print(".....")
hehe[0] = 101
print(hehe)
```

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
.....
[[101. 101. 101. 101.]
 [ 1.   1.   1.   1.]
 [ 1.   1.   1.   1.]
 [ 1.   1.   1.   1.]]
```

Broadcast Array: if
ones to change more
than one index value
at a time

```
[5]: #Array arithmetic operation
import numpy as RNA
_1array = RNA.array([10, 20, 30])
_2array = RNA.array([50, 60, 70])
result = _1array + _2array
print(result)
```

[60 80 100]

**Array arithmetic
operation**

→ perform -, *, / by
yourself

The **genfromtxt()** function is used to load data in a program from a text file. It takes multiple argument values to clean the data of the text file. It also has the ability to deal with missing or null values through the processes of filtering, removing, and replacing.

The **genfromtxt()** function from the Numpy module is perfect for **data loading and cleaning**.

From text/notepad/wordpad file

```
[27]: import numpy as RNA
      fileread = RNA.genfromtxt('rawdata.txt', delimiter=',')
      fileread

[27]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11., 12., 13.,
            14., 15., 16., 17., 18., 19., 20., 21., 22., 23., 23., 24., 25.,
            26., 27., 27., 29., 30.] )
```

```
[31]: import numpy as RNA
      fileread = RNA.genfromtxt('rawdata.txt', delimiter=',')
      print(fileread)
      print(".....")
      fileread = fileread.astype('int32')
      print(fileread)

[ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10. 11. 12. 13. 14. 15. 16. 17. 18.
 19. 20. 21. 22. 23. 23. 24. 25. 26. 27. 27. 29. 30.]
.....
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 23
 24 25 26 27 27 29 30]
```

Previous output type is float, hence convert into intger

```
import numpy as RNA
fileread = RNA.genfromtxt('rawchar.txt', dtype=str, encoding = None, delimiter=",")
for i in fileread:
    print(i,end=" ")
```

Bangladesh officially the People's Republic of Bangladesh is a country in South Asia. It is the eighth-most populous country in the world and among the most densely populated with a population of 170 million in an area of 148 460 square kilometres (57 320 sq mi). Bangladesh shares land borders with India to the north west and east and Myanmar to the southeast. To the south it has a coastline along the Bay of Bengal. It is separated from Bhutan and Nepal by the Siliguri Corridor and from China by the mountainous Indian state of Sikkim. Dhaka the capital and largest city is the nation's political financial and cultural centre. Chittagong is the second-largest city and the busiest port. The official language is Bengali with Bangladeshi English also used in government.

From CSV

```
[78]: from numpy import loadtxt
import numpy as RNA
csvread = open('csv1.csv','rb')
read = loadtxt(csvread, delimiter=',')
print(read)
```

```
[[ 1. 10. 20. 30.]
 [ 2. 11. 21. 31.]
 [ 3. 12. 22. 32.]
 [ 4. 13. 23. 33.]
 [ 5. 14. 24. 34.]
 [ 6. 15. 25. 35.]
 [ 7. 16. 26. 36.]
 [ 8. 17. 27. 37.]
 [ 9. 18. 28. 38.]
[10. 19. 29. 39.]]
```

loadtxt library file has to be imported if you want to load data from csv file. After that “csv” file will be accessed using open() function.

What is Pandas?

- Pandas is a Python library used for working with data sets
- It has functions for analyzing, cleaning, exploring, and manipulating data
- The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008

Why Use Pandas?

- Pandas allows us to **analyze big data and make conclusions based on statistical theories.**
- Pandas can **clean messy data sets, and make them readable** and relevant
- Relevant data is very **important in data science**

What Can Pandas Do?

Pandas gives you answers about the data. Like:

- Is there a correlation between two or more columns?
- What is average value?
- Max value?
- Min value?

Pandas are also able to **delete rows that are not relevant**, or contains wrong values, like empty or NULL values. This is called *cleaning* the data.

Data Cleaning

Data cleaning means fixing bad data in your data set.

Bad data could be:

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates

```
[5]: import pandas as RPD
rasheddataset = {
    'cars': ["Nissan", "Honda", "Ford", "Toyota"],
    'passings': [5, 6, 3, 4]
}
vovo = RPD.DataFrame(rasheddataset)
print(vovo)
```

	cars	passings
0	Nissan	5
1	Honda	6
2	Ford	3
3	Toyota	4

By default
index

```
[9]: #Create your own Labels:
import pandas as RPD
RM = [10, 17, 12, 23]
vovo = RPD.Series(RM, index = ["A", "B", "C", "D"])
print(vovo)
```

A	10
B	17
C	12
D	23

dtype: int64

Customized
index

```
[13]: # What is a DataFrame?  
# A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array, or a table with rows and columns.  
# Create a DataFrame from two Series:  
import pandas as RPD  
data = {  
    "Student_ID": [2024101, 2024102, 2024103],  
    "Height": [5, 5.5, 6]  
}  
#load data into a DataFrame object:  
vovo = RPD.DataFrame(data)  
print(vovo)
```

	Student_ID	Height
0	2024101	5.0
1	2024102	5.5
2	2024103	6.0

```
[15]: #to print specific row
import pandas as RPD
data = {
    "Student_ID": [2024101, 2024102, 2024103],
    "Height": [5, 5.5, 6]
}
#load data into a DataFrame object:
vovo = RPD.DataFrame(data)
print(vovo.loc[[0, 1]])
```

	Student_ID	Height
0	2024101	5.0
1	2024102	5.5

```
[17]: #index naming
import pandas as RPD
data = {
    "Student_ID": [2024101, 2024102, 2024103],
    "Height": [5, 5.5, 6]
}
vovo = RPD.DataFrame(data, index = ["i", "ii", "iii"])
print(vovo)
```

	Student_ID	Height
i	2024101	5.0
ii	2024102	5.5
iii	2024103	6.0

```
[25]: # Data read from CSV using panda
import pandas as RPD
vovo = RPD.read_csv('pandacsv1.csv')
print(vovo)
```

	Name	ID	Gender	Address
0	Rashed	101	M	Mirpur
1	Abir	102	M	Savar
2	Fatema	103	F	Banani
3	Robin	104	M	Paltan
4	Laila	105	F	Gulshan
5	Karim	106	M	Badda
6	Debashish	107	M	Kakoli
7	Naim	108	M	Dhanmondi
8	Farhad	109	M	Rampura

```
[27]: # How many rows at a time (60)
print(RPD.options.display.max_rows)
```

60

```
[29]: # view, for example first 5 rows
import pandas as RPD
vovo = RPD.read_csv('pandacsv1.csv')
print(vovo.head(5))
```

	Name	ID	Gender	Address
0	Rashed	101	M	Mirpur
1	Abir	102	M	Savar
2	Fatema	103	F	Banani
3	Robin	104	M	Paltan
4	Laila	105	F	Gulshan

```
[31]: # view last 5 rows
print(vovo.tail())
```

	Name	ID	Gender	Address
4	Laila	105	F	Gulshan
5	Karim	106	M	Badda
6	Debashish	107	M	Kakoli
7	Naim	108	M	Dhanmondi
8	Farhad	109	M	Rampura

```
[35]: # Return a new Data Frame with no empty cells  
# By default, the dropna() method returns a new DataFrame, and will not change the original.  
import pandas as RPD  
vovo = RPD.read_csv('panda_practice.csv')  
hehe = vovo.dropna()  
print(hehe.head(20))
```

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
5	60	102	127	300.0
6	60	110	136	374.0
7	45	104	134	253.3
8	30	109	133	195.1
9	60	98	124	269.0
10	60	103	147	329.3
11	60	100	120	250.7
12	60	106	128	345.3
13	60	104	132	379.3
14	60	98	123	275.0
15	60	98	120	215.2
16	60	100	120	300.0
18	60	103	123	323.0
19	45	97	125	243.0
20	60	108	131	364.2


```
[2]: import pandas as RPD
      vovo = RPD.read_csv('panda_edit.csv')
      vovo.fillna(130, inplace = True)
      print(vovo.head(20))
```

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
5	60	102	127	300.0
6	60	110	136	374.0
7	45	104	134	253.3
8	30	109	133	195.1
9	60	98	124	269.0
10	60	103	147	329.3
11	60	100	120	250.7
12	60	106	128	345.3
13	60	104	132	379.3
14	60	98	123	275.0
15	60	98	120	215.2
16	60	100	120	300.0
17	45	90	112	130.0
18	60	103	123	323.0
19	45	97	125	243.0

Another way of dealing with empty cells is to insert a new value instead.

The fillna() method allows us to replace all empty cells with a value

```
import pandas as RPD
vovo = RPD.read_csv('panda_edit.csv')
vovo.fillna({"Calories": 150}, inplace=True)
print(vovo.head(20))
```

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
5	60	102	127	300.0
6	60	110	136	374.0
7	45	104	134	253.3
8	30	109	133	195.1
9	60	98	124	269.0
10	60	103	147	329.3
11	60	100	120	250.7
12	60	106	128	345.3
13	60	104	132	379.3
14	60	98	123	275.0
15	60	98	120	215.2
16	60	100	120	300.0
17	45	90	112	150.0
18	60	103	123	323.0
19	45	97	125	243.0

**If you want to specify
any column**

```
import pandas as RPD
vovo = RPD.read_csv('panda_edit.csv')
RM = vovo["Calories"].mean()
vovo.fillna({"Calories": RM}, inplace=True)
print(vovo.head(20))
```

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.100000
1	60	117	145	479.000000
2	60	103	135	340.000000
3	45	109	175	282.400000
4	45	117	148	406.000000
5	60	102	127	300.000000
6	60	110	136	374.000000
7	45	104	134	253.300000
8	30	109	133	195.100000
9	60	98	124	269.000000
10	60	103	147	329.300000
11	60	100	120	250.700000
12	60	106	128	345.300000
13	60	104	132	379.300000
14	60	98	123	275.000000
15	60	98	120	215.200000
16	60	100	120	300.000000
17	45	90	112	375.790244
18	60	103	123	323.000000
19	45	97	125	243.000000

If you want to specify
any column and mean
value

Find median

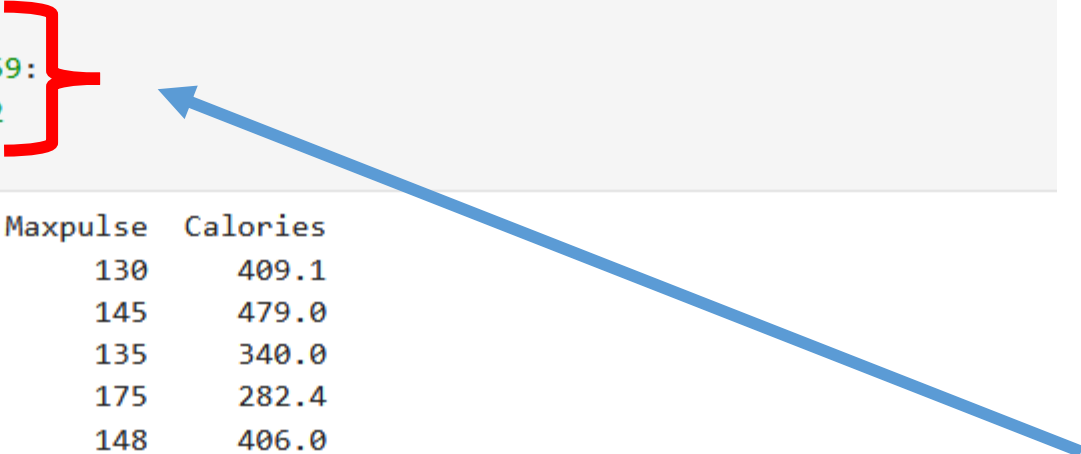
Find mode

```
#Wrong data --> Type mistake
import pandas as RPD
vovo = RPD.read_csv('data_wrong.csv')
vovo.loc[7:9, 'Duration'] = 29
print(vovo.head(20))
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	29-09-2024	110	130	409.1
1	60	30-09-2024	117	145	479.0
2	60	30-09-2024	103	135	340.0
3	45	30-09-2025	109	175	282.4
4	45	1/10/2024	117	148	406.0
5	60	30-09-2026	102	127	300.0
6	60	2/10/2024	110	136	374.0
7	29	30-09-2027	104	134	253.3
8	29	3/10/2024	109	133	195.1
9	29	30-09-2028	98	124	269.0
10	60	4/10/2024	103	147	329.3
11	60	30-09-2029	100	120	250.7
12	60	5/10/2024	106	128	345.3
13	60	30-09-2030	104	132	379.3

**Try to find out wrong
value and make
correction**

```
#Wrong data --> Type mistake ---> if the data set is large enough --> need to set condition
import pandas as RPD
vovo = RPD.read_csv('data_wrong.csv')
vovo.loc[7:9, 'Duration'] = 29
for rm in vovo.index:
    if vovo.loc[rm, "Duration"] > 59:
        vovo.loc[rm, "Duration"] = 22
print(vovo.head(20))
```



	Duration	Date	Pulse	Maxpulse	Calories
0	22	29-09-2024	110	130	409.1
1	22	30-09-2024	117	145	479.0
2	22	30-09-2024	103	135	340.0
3	45	30-09-2025	109	175	282.4
4	45	1/10/2024	117	148	406.0
5	22	30-09-2026	102	127	300.0
6	22	2/10/2024	110	136	374.0
7	29	30-09-2027	104	134	253.3
8	29	3/10/2024	109	133	195.1
9	29	30-09-2028	98	124	269.0
10	22	4/10/2024	103	147	329.3
11	22	30-09-2029	100	120	250.7
12	22	5/10/2024	106	128	345.3
13	22	30-09-2030	104	132	379.3
14	22	6/10/2024	98	123	275.0
15	22	30-09-2031	98	120	215.2
16	22	7/10/2024	100	120	300.0
17	45	30-09-2032	90	112	NaN
18	22	8/10/2024	103	123	323.0
19	45	30-09-2033	97	125	243.0

**Try to find out wrong
value and make
correction**

**** based on certain
condition**

```
import pandas as RPD
vovo = RPD.read_csv('data_wrong.csv')
vovo.loc[7:9, 'Duration'] = 29
for rm in vovo.index:
    if vovo.loc[rm, "Duration"] > 59:
        vovo.drop(rm, inplace = True)
print(vovo)
```

	Duration	Date	Pulse	Maxpulse	Calories
3	45	30-09-2025	109	175	282.4
4	45	1/10/2024	117	148	406.0
7	29	30-09-2027	104	134	253.3
8	29	3/10/2024	109	133	195.1
9	29	30-09-2028	98	124	269.0
..	...	...	...	...	...
159	30	30-09-2103	80	120	240.9
160	30	18-12-2024	85	120	250.4
161	45	30-09-2104	90	130	260.4
162	45	19-12-2024	95	130	270.0
163	45	30-09-2105	100	140	280.9

[64 rows x 5 columns]

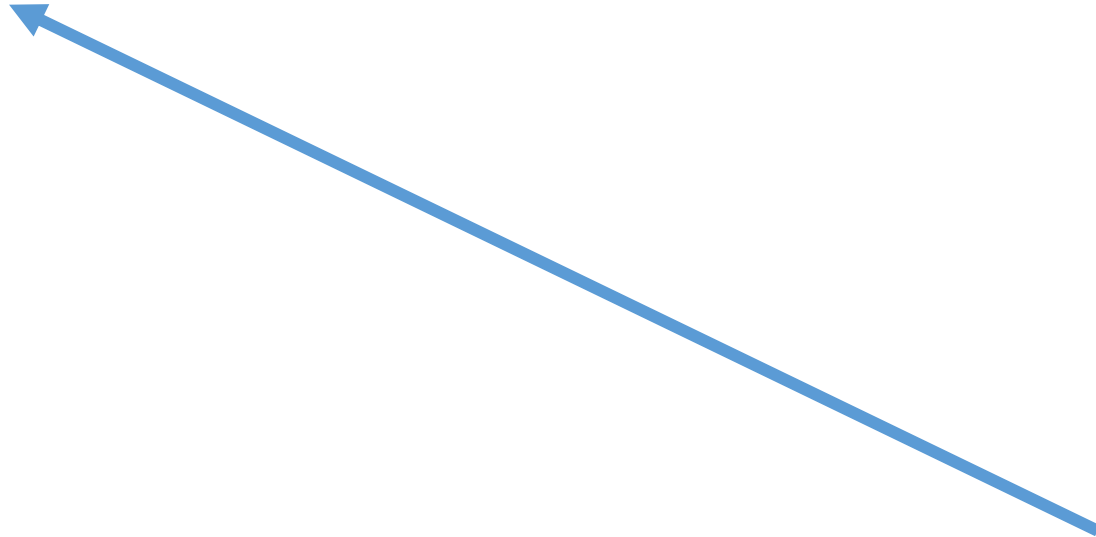
Drop the rows
Based on
Certain condition


```
import pandas as RPD
vovo = RPD.read_csv('data_wrong.csv')
hehe = vovo.ffill(axis = 0)
print(hehe.head(20))
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60.0	29-09-2024	110	130	409.1
1	60.0	30-09-2024	117	145	473.0
2	60.0	30-09-2024	103	135	340.0
3	45.0	30-09-2025	109	175	282.4
4	45.0	1/10/2024	117	148	406.0
5	45.0	30-09-2026	102	127	300.0
6	60.0	2/10/2024	110	136	374.0
7	373.0	30-09-2027	104	134	253.3
8	30.0	3/10/2024	109	133	195.1
9	523.0	30-09-2028	98	124	269.0
10	60.0	4/10/2024	103	147	329.3
11	60.0	30-09-2029	100	120	250.7

**Fill using
forward value**

?



**Fill using
backward value**

Handling of Duplicates value

→ The duplicated() method returns a Boolean values for each row

For example,

- Returns **True for every row** that is a duplicate, otherwise **False**

handling of duplicate value (**current data set view**)

```
import pandas as RPD
```

```
vovo = RPD.read_csv('panda_edit.csv')
```

```
print(vovo.head(40))
```



5	60	102	127	300.0
6	60	110	136	374.0
7	45	104	134	253.3
8	30	109	133	195.1
9	60	98	124	269.0
10	60	103	147	329.3
11	60	100	120	250.7
12	60	106	128	345.3
13	60	104	132	379.3
14	60	98	123	275.0
15	60	98	120	215.2
16	60	100	120	300.0
17	45	90	112	NaN
18	60	103	123	323.0
19	45	97	125	243.0
20	60	108	131	364.2
21	45	100	119	282.0
22	60	130	101	300.0
23	45	105	132	246.0
24	60	102	126	334.5
25	60	100	120	250.0
26	60	92	118	241.0
27	60	103	132	NaN
28	60	100	132	280.0
29	60	102	129	380.3
30	60	92	115	243.0
31	45	90	112	180.1
32	60	101	124	299.0
33	60	93	113	223.0
34	60	107	136	361.0
35	60	114	140	415.0
36	60	102	127	300.0
37	60	100	120	300.0
38	60	100	120	300.0
39	45	104	129	266.0



[8]: *# handling of duplicate value (boolean value, true for duplicate)*

```
import pandas as RPD
vovo = RPD.read_csv('panda_edit.csv')
popo = vovo.duplicated()
print(popo.head(40))
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9    False
10   False
11   False
12   False
13   False
14   False
15   False
16   False
```

```
# handling of duplicate value (delete duplicate values)
import pandas as RPD
bobo = RPD.read_csv('panda_edit.csv')
bobo.drop_duplicates(inplace = True)
print(bobo.head(40))
```

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0



28	60	100	132	280.0
29	60	102	129	380.3
30	60	92	115	243.0
31	45	90	112	180.1
32	60	101	124	299.0
33	60	93	113	223.0
34	60	107	136	361.0
35	60	114	140	415.0
39	45	104	129	266.0
41	60	98	126	286.0
42	60	100	122	329.4
43	60	111	138	400.0

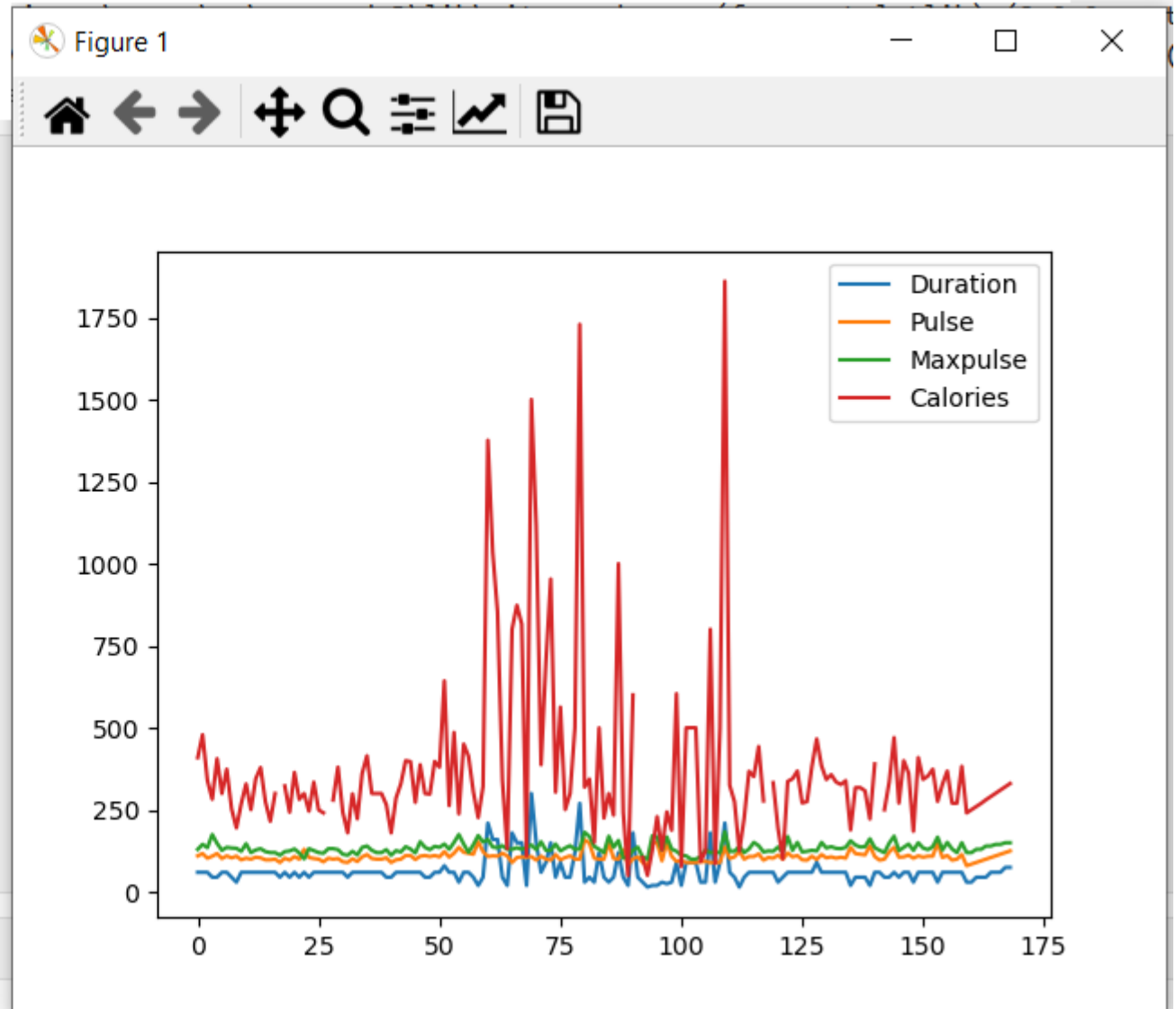


Plotting:

Pandas uses the plot() method to create diagrams.

```
# pandas: graphical representation  
# type, pip install matplotlib  
# type, conda install -c anaconda pyqt  
# type, pip install PyQt6
```

```
import sys  
import matplotlib  
matplotlib.use('Qt5Agg')  
  
import pandas as RPD  
import matplotlib.pyplot as PLOT  
  
vovo = RPD.read_csv('panda_edit.csv')  
  
vovo.plot()  
  
PLOT.show()
```



Plotting:

Pandas uses the plot()
scatter graph-> two columns based

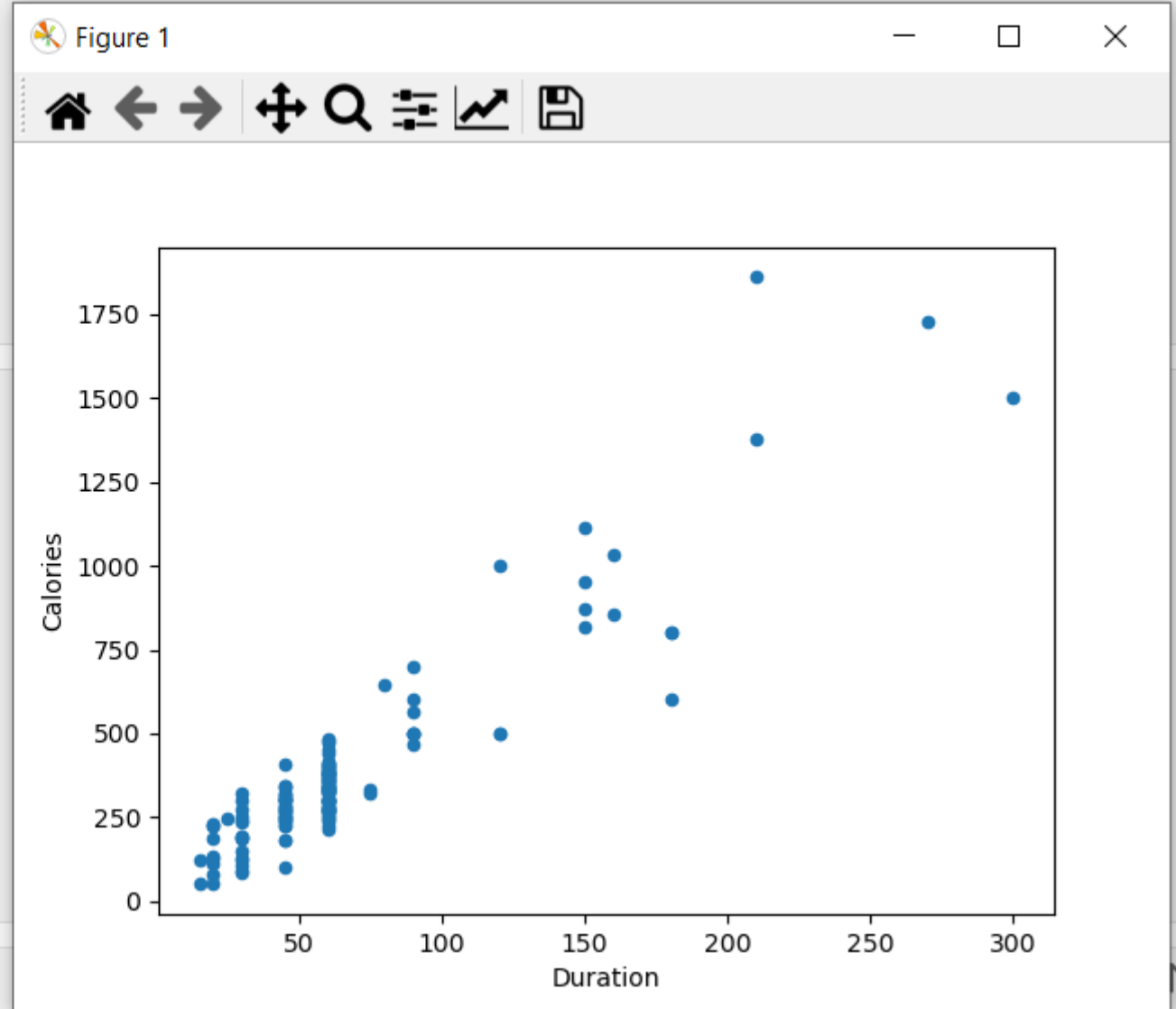
```
# for particular columns --> scatter graph
import sys
import matplotlib
matplotlib.use('Qt5Agg')

import pandas as RPD
import matplotlib.pyplot as PLT

vovo = RPD.read_csv('panda_edit.csv')

vovo.plot(kind = 'scatter', x = 'Duration', y = 'Calories')

PLT.show()
```



Plotting:

Pandas uses the plot()

histogram:

based on single column

```
#histogram based on single column
import sys
import matplotlib
matplotlib.use('Qt5Agg')

import pandas as RPD
import matplotlib.pyplot as PLT

vovo = RPD.read_csv('panda_edit.csv')

vovo["Calories"].plot(kind = 'hist')

PLT.show()
```

